`mdarray`: An Owning Multidimensional Array Analog of `mdspan`

Document #:	P1684R1
Date:	2022-03-15
Project:	Programming Language C++ Library Evolution

Reply-to:	Christian Trott <crtrott@sandia.gov> David Hollman <me@dsh.fyi> Mark Hoemmen <mhoemmen@stellarscience.com> Daniel Sunderland <dansunderland@gmail.com>

Contents

1 Revision History

1.1 P1684r1: 2022-03 Mailing

1.1.0.1 Changes from R0

- harmonize with P0009R15
- don't use ContainerPolicy, simply use Container as fourth template argument
- added wording

2 Motivation

[P0009R9] introduced a non-owning multidimensional array abstraction that has been refined over many revisions and is expected to be merged into the C++ working draft early in the C++23 cycle. However, there are many owning use cases where `mdspan` is not ideal. In particular, for use cases with small, fixed-size dimensions, the non-owning semantics of `mdspan` may represent a significant pessimization, precluding optimizations that arise from the removal of the non-owning indirection (such as storing the data in registers).

Without `mdarray`, use cases that should be owning are awkward to express:

P0009 Only:

```
void make_random_rotation(mdspan<float, 3, 3> output);
void apply_rotation(mdspan<float, 3, 3>, mdspan<float, 3>);
void random_rotate(mdspan<float, dynamic_extent, 3> points) {
  float buffer[9] = { };
  auto rotation = mdspan<float, 3, 3>(buffer);
  make_random_rotation(rotation);
  for(int i = 0; i < points.extent(0); ++i) {
    apply_rotation(rotation, subspan(points, i, std::all));
  }
}
```

This Work:

```
mdarray<float, 3, 3> make_random_rotation();
void apply_rotation(mdarray<float, 3, 3>&, const mdspan<float, 3>&);
void random_rotate(mdspan<float, dynamic_extent, 3> points) {
  auto rotation = make_random_rotation();
  for(int i = 0; i < points.extent(0); ++i) {
    apply_rotation(rotation, subspan(points, i, std::all));
  }
}
```

Particularly for small, fixed-dimension `mdspan` use cases, owning semantics can be a lot more convenient and require a lot less in the way of interprocedural analysis to optimize.

§ 3 Design

One major goal of the design for `mdarray` is to parallel the design of `mdspan` as much as possible (but no more), with the goals of reducing cognitive load for users already familiar with `mdspan` and of incorporating the lessons learned from the half decade of experience on [P0009R9]. This paper (and this section in particular) assumes the reader has read and is already familiar with [P0009R9].

§ 3.1 Design Overview

In brief, the analogy to `mdspan` can be seen in the declaration of the proposed design for `mdarray`:

```
template<class ElementType,
         class Extents,
         class LayoutPolicy = layout_right,
         class Container = see-below>
  class mdarray;
```

This intentionally parallels the design of `mdspan` in [P0009R9], which has the signature:

```
template<class ElementType,
         class Extents,
         class LayoutPolicy = layout_right,
         class AccessorPolicy = accessor_basic<ElementType>>
  class mdspan;
```

Note that the original proposal for `mdarray` had a `ContainerPolicy` instead of a `Container` parameter. We decided that the complexity of a ContainerPolicy is not actually required for `mdarray`. The vast majority of cases where customization of the `AccessorPolicy` is required to modify the access behavior, are local contextual requirements. For example one has a loop with write conflicts, and wants an atomic accessor. However, even with `mdarray` existing these temporary objects need to be views of data, and thus one should create an `mdspan` view with appropriate access behavior from the `mdarray` which owns the data.

§ 3.2 Principal Differences between `mdarray` and `mdspan`

By design, `mdarray` is as similar as possible to `mdspan`, except with container semantics instead of reference semantics. However, the use of container semantics necessitates a few differences. The most notable of these is deep `constness`. Like all reference semantic types in the standard, `mdspan` has shallow `constness`, but container types in the standard library propagate `const` through their access functions. Thus, `mdarray` needs `const` and non-`const` versions of every analogous operation in `mdspan` that interacts with the underlying data:

```
template<class ElementType, class Extents, class LayoutPolicy, class
        Container>
class mdarray {
  /* ... */

  // also in mdspan:
  using pointer = /* ... */;
  // only in mdarray:
  using const_pointer = /* ... */;

  // also in mdspan:
  using reference = /* ... */;
  // only in mdarray:
  using const_reference = /* ... */;

  // analogous to mdspan, except with const_reference return type:
  template<class... IndexType>
    constexpr const_reference operator[](IndexType...) const;
  template<class IndexType, size_t N>
    constexpr const_reference operator[](const array<IndexType, N>&)
        const;
  // non-const overloads only in mdarray:
  template<class... IndexType>
    constexpr reference operator[](IndexType...);
  template<class IndexType, size_t N>
    constexpr reference operator[](const array<IndexType, N>&);

  // also in mdspan, except with const_pointer return type:
  constexpr const_pointer data() const noexcept;
  // non-const overload only in mdarray:
  constexpr pointer data() noexcept;

  /* ... */
};
```

Additionally, `mdarray` needs a means of interoperating with `mdspan` in roughly the same way as contiguous containers interact with `span`, or as `string` interacts with `string_view`. We could do this by adding a constructor to `mdspan`, which would be more consistent with the analogous features in `span` and `string_view`, but in the interest of avoiding modifications to an in-flight proposal, we propose using a member function of `mdarray` for this functionality for now (tentatively named `view()`, subject to bikeshedding). We are happy to change this based on design direction from LEWG:

```
template<class ElementType, class Extents, class LayoutPolicy, class
        Container>
class mdarray {
  /* ... */

  // only in mdarray:
  using view_type = /* ... */;
  using const_view_type = /* ... */;
  view_type view() noexcept;
  const_view_type view() const noexcept;
```

```
  /* ... */
};
```

As discussed below, `accessor_policy` from `mdspan` is replaced by `container_policy` in `mdarray`:

```
template<class ElementType, class Extents, class LayoutPolicy, class
        ContainerPolicy>
class mdarray {
  /* ... */

  // only in mdarray:
  using container_policy_type = ContainerPolicy;
  using container_type = /* ... */;

  /* ... */
};
```

3.2.0.1 Constructors and Assignment Operators

There are several relatively trivial differences between `mdspan` and `mdarray` constructors and assignment operators. Most trivially, `mdspan` provides a compatible `mdspan` copy-like constructor and copy-like assignment operator, with proper constraints and expectations to enforce compatibility of shape, layout, and size. Since `mdarray` has owning semantics, we also need move-like versions of these:

```
template<class ElementType, class Extents, class LayoutPolicy, class
        ContainerPolicy>
class mdarray {
  /* ... */

  // analogous to mdspan:
  template<class ET, class Exts, class LP, class CP>
    constexpr mdarray(const mdarray<ET, Exts, LP, CP>&);
  template<class ET, class Exts, class LP, class CP>
    constexpr mdarray& operator=(const mdarray<ET, Exts, LP, CP>&);
  // only in mdarray:
  template<class ET, class Exts, class LP, class CP>
    constexpr mdarray(mdarray<ET, Exts, LP, CP>&&) noexcept(see-below);
  template<class ET, class Exts, class LP, class CP>
    constexpr mdarray& operator=(mdarray<ET, Exts, LP, CP>&&) noexcept;

  /* ... */
};
```

(The `noexcept` clauses on these constructors and operators should probably actually derive from `noexcept` clauses on the analogous functionality for the element type and policy types).

Additionally, the analog of the `mdspan(pointer, IndexType...)` constructor for `mdarray` should not take the first argument, since the `mdarray` owns the data and thus should be able to construct it from sizes:

```
template<class ElementType, class Extents, class LayoutPolicy, class
        ContainerPolicy>
class mdarray {
  /* ... */

  // only in mdarray
  template <class... IndexType>
  explicit constexpr mdarray(IndexType...);

  /* ... */
};
```

Note that in the completely static extents case, this is ambiguous with the default constructor. For consistency in generic code, the semantics of this constructor should be preferred over those of the default constructor in that case.

By this same logic, we arrive at the `mapping_type` constructor analogs:

```
template<class ElementType, class Extents, class LayoutPolicy, class
        Container>
class mdarray {
  /* ... */

  // only in mdarray
  explicit constexpr mdarray(const mapping_type&);

  /* ... */
};
```

There is some question as to whether we should also have constructors that take `container_type` instances in addition to indices. Consistency with standard container adapters like `std::priority_queue` would dictate that we should. Note that these constructors would copy data over.

```
template<class ElementType, class Extents, class LayoutPolicy, class
        Container>
class mdarray {
  /* ... */

  // only in mdarray
  explicit constexpr mdarray(const mapping_type&);
  constexpr mdarray(const container_type&, const mapping_type&);

  /* ... */
};
```

Finally, we remmove the constructor that takes an `array<IndexType, N>` of dynamic extents because of the ambiguity or confusion with a constructor that takes a container instance in the case where the `container` happens to be an `array<IndexType, N>`.

§ 3.3 `Extents` Design Reused

As with `mdspan`, the `Extents` template parameter to `mdarray` shall be a template instantiation of `std::extents`, as described in [P0009R9]. The concerns addressed by this aspect of the design are exactly the same in `mdarray` and `mdspan`, so using the same form and mechanism seems like the right thing to do here.

3.4 `LayoutPolicy` Design Reused

While not quite as straightforward, the decision to use the same design for `LayoutPolicy` from `mdspan` in `mdarray` is still quite obviously the best choice. The only piece that's a bit of a less perfect fit is the `is_contiguous()` and `is_always_contiguous()` requirements. While non-contiguous use cases for `mdspan` are quite common (e.g., `subspan()`), non-contiguous use cases for `mdarray` are expected to be a bit more arcane. Nonetheless, reasonable use cases do exist (for instance, padding of the fast-running dimension in anticipation of a resize operation), and the reduction in cognitive load due to concept reuse certainly justifies reusing `LayoutPolicy` for `mdarray`.

3.5 `AccessorPolicy` Replaced by `Container`

By far the most complicated aspect of the design for `mdarray` is the analog of the `AccessorPolicy` in `mdspan`. The `AccessorPolicy` for `mdspan` is clearly designed with non-owning semantics in mind–it provides a `pointer` type, a `reference` type, and a means of converting from a pointer and an offset to a reference. Beyond the lack of an allocation mechanism (that would be needed by `mdarray`), the `AccessorPolicy` requirements address concerns normally addressed by the allocation mechanism itself. For instance, the C++ named requirements for `Allocator` allow for the provision of the `pointer` type to `std::vector` and other containers. Arguably, consistency between `mdarray` and standard library containers is far more important than with `mdspan` in this respect. Several approaches to addressing this incongruity are discussed below.

3.5.1 Expected Behavior of Motivating Use Cases

Regardless of the form of the solution, there are several use cases where we have a clear understanding of how we want them to work. As alluded to above, perhaps *the* most important motivating use case for `mdarray` is that of small, fixed-size extents. Consider a fictitious (not proposed) function, *get-underlying-container*, that somehow retrieves the underlying storage of an `mdarray`. For an `mdarray` of entirely fixed sizes, we would expect the default implementation to return something that is (at the very least) convertible to `array` of the correct size:

```
auto a = mdarray<int, 3, 3>();
std::array<int, 9> data = get-underlying-container(a);
```

(Whether or not a reference to the underlying container should be obtainable is slightly less clear, though we see no reason why this should not be allowed.) The default for an `mdarray` with variable extents is only slightly less clear, though it should almost certainly meet the requirements of *contiguous container* (**[container.requirements.general]**/13).

The default model for *contiguous container* of variable size in the standard library is `vector`, so an entirely reasonable outcome would be to have:

```
auto a = mdarray<int, 3, dynamic_extent>();
std::vector<int> data = get-underlying-container(a);
```

Moreover, taking a view of a `mdarray` should yield an analogous `mdspan` with consistent semantics (except, of course, that the latter is non-owning). We provisionally call the method for obtaining this analog `view()`:

```
template <class T, class Extents, class LayoutPolicy, class
        ContainerPolicy>
void frobnicate(mdarray<T, Extents, LayoutPolicy, ContainerPolicy> data)
{
  auto data_view = data.view();
  /* ... */
}
```

In order for this to work, `Container::data()` should be required to return `T*`. That way interoperability with `mdspan` is trivial, since it can simply be created as:

```
template <class T, class Extents, class LayoutPolicy, class
        ContainerPolicy>
void frobnicate(mdarray<T, Extents, LayoutPolicy, ContainerPolicy> a)
{
  mdspan a_view(a.data(), a.mapping());
  /* ... */
}
```

§ 3.5.2 Analogs in the Standard Library: Container Adapters

Perhaps the best analogs for what `mdarray` is doing with respect to allocation and ownership are the container adaptors (**[container.adaptors]**), since they imbue additional semantics to what is otherwise an ordinary container. These all take a `Container` template parameter, which defaults to `deque` for `stack` and `queue`, and to `vector` for `priority_queue`. The allocation concern is thus delegated to the container concept, reducing the cognitive load associated with the design. While this design approach overconstrains the template parameter slightly (i.e., not all of the requirements of the `Container` concept are needed by the container adaptors), the simplicity arising from concept reuse more than justifies the cost of the extra constraints.

It is difficult to say whether the use of `Container` directly, as with the container adaptors, is also the correct approach for `mdarray`. There are pieces of information that may need to be customized in some very reasonable use cases that are not provided by the standard container concept. The most important of these is the ability to produce a semantically consistent `AccessorPolicy` when creating a `mdspan` that refers to a `mdarray`. (Interoperability between `mdspan` and `mdarray` is considered a critical design requirement because of the nearly complete overlap in the set of algorithms that operate on them.) For instance, given a `Container` instance `c` and an `AccessorPolicy` instance `a`, the behavior of `a.access(p, n)` should be consistent with the behavior of `c[n]` for a `mdspan` wrapping

a that is a view of a `mdarray` wrapping `c` (if `p` is `c.begin()`). But because `c[n]` is part of the container requirements and thus may encapsulate any arbitrary mapping from an offset of `c.begin()` to a reference, the only reasonable means of preserving these semantics for arbitrary container types is to reference the original container directly in the corresponding `AccessorPolicy`. In other words, the signature for the `view()` method of `mdarray` would need to look something like (ignoring, for the moment, whether the name for the type of the accessor is specified or implementation-defined):

```
template<class ElementType,
         class Extents,
         class LayoutPolicy,
         class Container>
struct mdarray {
 /* ... */
 mdspan<
   ElementType, Extents, LayoutPolicy,
   container_reference_accessor<Container>>
 view() const noexcept;
 /* ... */
};

template <class Container>
struct __container_reference_accessor { // not proposed
  using pointer = Container*;
  /* ... */
  template <class Integer>
  reference access(pointer p, Integer offset) {
    return (*p)[offset];
  }
  /* ... */
};
```

But this approach comes at the cost of an additional indirection (one for the pointer to the container, and one for the container dereference itself), which is likely unacceptable cost in a facility designed to target performance-sensitive use cases. The situation for the `offset` requirement (which is used by `subspan`) is potentially even worse for arbitrary non-contiguous containers, adding up to one indirection per invocation of `subspan`. This is likely unacceptable in many contexts.

Nonetheless, using refinements of the existing `Container` concept directly with `mdarray` is an incredibly attractive option because it avoids the introduction of an extra concept, and thus significantly decreases the cognitive cost of the abstraction. Thus, direct use of the existing `Container` concept hierarchy should be preferred to other options unless the shortcomings of the existing concept are so irreconcilable (or so complicated to reconcile) as to create more cognitive load than is needed for an entirely new concept.

One straight forward way to resolve the above concerns with arbitrary container types, is to simply restrict what type of containers can be used.

Specifically we would restrict it such that creating an `mdspan` with `default_accessor` is straight forward. Thus we would require:

```
std::is_same_v<decltype(Container::data()),ElementType*> == true
```

and

```
&c[i] == c.data()+i
```

for all `i` in the range of `[0,c.size())`.

In the following we discuss two design alternatives.

3.5.3 (Not Proposed) Alternative: A Dedicated `ContainerPolicy` Concept

Despite the additional cognitive load, there are a few arguments in favor of using a dedicated concept for the container description of `mdarray`. As is often the case with concept-driven design, the implementation of `mdarray` only needs a relatively small subset of the interface elements in the `Container` concept hierarchy. This alone is not enough to justify an additional concept external to the existing hierarchy; however, there are also quite a few features missing from the existing container concept hierarchy, without which an efficient `mdarray` implementation may be difficult or impossible. As alluded to above, conversion to an `AccessorPolicy` for the creation of a `mdspan` is one missing piece. (Another, interestingly, is sized construction of the container mixed with allocator awareness, which is surprisingly lacking in the current hierarchy somehow.) For these reasons, it is worth exploring a design based on analogy to the `AccessorPolicy` concept rather than on analogy to `Container`. If we make that abstraction owning, we might call it something like `_ContainerLikeThing` (not proposed here; included for discussion). In that case, a model of the `_ContainerLikeThing` concept that meets the needs of `mdarray` might look something like:

```
template <class ElementType, class Allocator=std::allocator<ElementType>>
struct vector_container_like_thing // models _ContainerLikeThing
{
public:
  using element_type = ElementType;
  using container_type = std::vector<ElementType, Allocator>;
  using allocator_type = typename container_type::allocator_type;
  using pointer = typename container_type::pointer;
  using const_pointer = typename container_type::const_pointer;
  using reference = typename container_type::reference;
  using const_reference = typename container_type::const_reference;
  using accessor_policy = std::accessor_basic<element_type>;
  using const_accessor_policy = std::accessor_basic<const element_type>;

  // analogous to `access` method in `AccessorPolicy`
  reference access(ptrdiff_t offset) { return __c[size_t(offset)]; }
  const_reference access(ptrdiff_t offset) const { return
        __c[size_t(offset)]; }

  // Interface for mdspan creation
  accessor_policy make_accessor_policy() { return { }; }
  const_accessor_policy make_accessor_policy() const { return { }; }
  typename pointer data() { return  __c.data(); }
```

```
  typename const_pointer data() const { return  __c.data(); }

  // Interface for sized construction
  static vector_container_policy create(size_t n) {
    return vector_container_like_thing{container_type(n, element_type{})};
  }
  static vector_container_policy create(size_t n, allocator_type const&
        alloc) {
    return vector_container_like_thing{container_type(n, element_type{},
        alloc)};
  }

  container_type __c;
};
```

This approach solves many of the problems associated with using the `Container` concept directly. It is the most flexible and provides the best compatibility with `mdspan`, since the conversion to analogous `AccessorPolicy` is fully customizable. This comes at the cost of additional cognitive load, but this can be justified based on the observation that almost half of the functionality in the above sketch is absent from the container hierarchy: the `make_accessor_policy()` requirement and the sized, allocator-aware container creation (`create(n, alloc)`) have no analogs in the container concept hierarchy. Non-allocator-aware creation (`create(n)`) is analogous to sized construction from the sequence container concept, the `data()` method is analogous to `begin()` on the contiguous container concept, and `access(n)` is analogous to `operator[]` or `at(n)` from the optional sequence container requirements. Even for these latter pieces of functionality, though, we are required to combine several different concepts from the `Container` hierarchy. Based on this analysis, we have decided it is reasonable to pursue designs for this customization point that diverge from `Container`, including ones that use `AccessorPolicy` as a starting point. Given a better design, we would definitely consider reversing direction on this decision, but despite significant effort, we were unable to find a design that was more than an awkward and forced fit for the `Container` concept hierarchy.

§ 3.5.4 (Not Proposed) Alternative: `ContainerPolicy` subsumes `AccessorPolicy`

The above approach has the significant drawback that the `_ContainerLikeThing` is an owning abstraction fairly similar to a container that diverges from the `Container` hierarchy. We initially explored this direction because it avoids having to provide a `mdarray` constructor that takes both a `Container` and a `ContainerPolicy`, which we felt was a “design smell.” Another alternative along these lines is to make the `mdarray` itself own the container instance and have the `ContainerPolicy` (name subject to bikeshedding; maybe `ContainerFactory` or `ContainerAccessor` is more appropriate?) be a non-owning abstraction that describes the container creation and access. While this approach leads to an ugly `mdarray(container_type, mapping_type, ContainerPolicy)` constructor, the analog that constructor affords to the `mdspan(pointer, mapping_type, AccessorPolicy)` constructor is a reasonable argument in favor of this design despite its quirkiness. Furthermore, this approach affords the opportunity to explore a `ContainerPolicy` design that subsumes `AccessorPolicy`, thus providing the needed

conversion to `AccessorPolicy` for the analogous `mdspan` by simple subsumption. More importantly, this subsumption would significantly decrease the cognitive load for users already familiar with `mdspan`. A model of `ContainerPolicy` for this sort of approach might look something like:

```
template <class ElementType, class Allocator=std::allocator<ElementType>>
struct vector_container_policy // models ContainerPolicy (and thus
        AccessorPolicy)
{
public:
  using element_type = ElementType;
  using container_type = std::vector<ElementType, Allocator>;
  using allocator_type = typename container_type::allocator_type;
  using pointer = typename container_type::pointer;
  using const_pointer = typename container_type::const_pointer;
  using reference = typename container_type::reference;
  using const_reference = typename container_type::const_reference;
  using offset_policy = vector_container_policy<ElementType, Allocator>

  // ContainerPolicy requirements:
  reference access(container_type& c, ptrdiff_t i) { return c[size_t(i)];
        }
  const_reference access(container_type const& ptrdiff_t i) const { return
        c[size_t(i)]; }

  // ContainerPolicy requirements (interface for sized construction):
  container_type create(size_t n) {
    return container_type(n, element_type{});
  }
  container_type create(size_t n, allocator_type const& alloc) {
    return container_type(n, element_type{}, alloc);
  }

  // AccessorPolicy requirement:
  reference access(pointer p, ptrdiff_t i) { return p[i]; }
  // For the const analog of AccessorPolicy:
  const_reference access(const_pointer p, ptrdiff_t i) const { return
        p[i]; }

  // AccessorPolicy requirement:
  pointer offset(pointer p, ptrdiff_t i) { return p + i; }
  // For the const analog of AccessorPolicy:
  const_pointer offset(const_pointer p, ptrdiff_t i) const { return p + i;
        }

  // AccessorPolicy requirement:
  element_type* decay(pointer p) { return p; }
  // For the const analog of AccessorPolicy:
  const element_type* decay(pointer p) const { return p; }

};
```

The above sketch makes clear the biggest challenge with this approach: the mismatch in shallow versus deep `const`ness in for an abstractions designed to support `mdspan` and `mdarray`, respectively. The `ContainerPolicy` concept thus requires additional `const`-

qualified overloads of the basis operations. Moreover, while the `ContainerPolicy` itself can be obtained directly from the corresponding `AccessorPolicy` in the case of the non-`const` method for creating the corresponding `mdspan` (provisionally called `view()`), the `const`-qualified version needs to adapt the policy, since the nested types have the wrong names (e.g., `const_pointer` should be named `pointer` from the perspective of the `mdspan` that the `const`-qualified `view()` needs to return). This could be fixed without too much mess using an adapter (that does not need to be part of the specification):

```
template <ContainerPolicy P>
class __const_accessor_policy_adapter { // models AccessorPolicy
public:
  using element_type = add_const_t<typename P::element_type>;
  using pointer = typename P::const_pointer;
  using reference = typename P::const_reference;
  using offset_policy = __const_accessor_policy_adapter<typename
        P::offset_policy>;

  reference access(pointer p, ptrdiff_t i) { return acc_.access(p, i); }
  pointer offset(pointer p, ptrdiff_t i) { return acc_.offset(p, i); }
  element_type* decay(pointer p) { return acc_.decay(p); }

private:
  [[no_unique_address]] add_const_t<P> acc_;
};
```

Compared to simply using a `container` as the argument, this approach has the benefit of enabling `mdarray` to use containers for which `data()[i]` is not giving access to the same element as `container[i]`. However, after more consideration we believe that the need for supporting such containers as underlying storage for `mdarray` is likely fairly niche. Furthermore, we believe one could later extent the design of `mdarray` to allow for such ContainerPolicies, even if the initial design only allows for a restricted set of containers.

§ 4 Wording

22.7.[illegible] Class template `mdarray` [mdarray]

22.7.[illegible].1 `mdarray` overview [mdarray.overview]

1 `mdarray` maps a multidimensional index in its domain to a reference to an element in its codomain.

2 The *domain* of an `mdarray` object is a multidimensional index space defined by an `extents`.

3 The *codomain* of an `mdarray` object is a set of elements accessible from a contiguous range of integer indices.

```
namespace std {
```

```
template<class ElementType, class Extents, class LayoutPolicy, class
        Container =
         see below>
class mdarray {
public:
  using extents_type = Extents;
  using layout_type = LayoutPolicy;
  using container_type = Container;
  using mapping_type = typename layout_type::template
        mapping<extents_type>;
  using element_type = ElementType;
  using value_type = element_type;
  using size_type = typename Extents::size_type ;
  using pointer = typename container_type::pointer;
  using reference = typename container_type::reference;
  using const_pointer = typename container_type::const_pointer;
  using const_reference = typename container_type::const_reference;

  // [mdspan.mdspan.ctors], mdspan Constructors
  constexpr mdarray() requires(rank_dynamic() != 0) = default;
  constexpr mdarray(const mdarray& rhs) = default;
  constexpr mdarray(mdarray&& rhs) = default;

  template<class... SizeTypes>
    requires(rank()>0 || rank_dynamic()==0)
    explicit constexpr mdarray(SizeTypes... exts);
  constexpr mdarray(const Extents& ext);
  constexpr mdarray(const mapping_type& m);


  template<class... SizeTypes>
    explicit constexpr mdarray(const container_type& c, SizeTypes...
        exts);
  constexpr mdarray(const container_type& c, const extents_type& ext);
  constexpr mdarray(const container_type& c, const mapping_type& m);

  template<class... SizeTypes>
    explicit constexpr mdarray(container_type&& c, SizeTypes... exts);
  constexpr mdarray(container_type&& c, const extents_type& ext);
  constexpr mdarray(container_type&& c, const mapping_type& m);

  template<class... SizeTypes>
    explicit constexpr mdarray(initializer_list<T> init, SizeTypes...
        exts);
  constexpr mdarray(initializer_list<T> init, const extents_type& ext);
  constexpr mdarray(initializer_list<T> init, const mapping_type& m);

  template<class OtherElementType, class OtherExtents,
           class OtherLayoutPolicy, class OtherContainer>
    explicit(see below)
    constexpr mdarray(
      const mdarray<OtherElementType, OtherExtents,
                    OtherLayoutPolicy, OtherContainer>& other);

  template<class Alloc, class... SizeTypes>
    requires(uses_allocator_v<container_type, Alloc>)
```

```
  constexpr mdarray(const Extents& ext, const Alloc& a);
template<class Alloc>
  requires(uses_allocator_v<container_type, Alloc>)
  constexpr mdarray(const mapping_type& m, const Alloc& a);

template<class Alloc>
  requires(uses_allocator_v<container_type, Alloc>)
  constexpr mdarray(const container_type& c, const extents_type& ext);
template<class Alloc>
  requires(uses_allocator_v<container_type, Alloc>)
  constexpr mdarray(const container_type& c, const mapping_type& m);

template<class Alloc>
  requires(uses_allocator_v<container_type, Alloc>)
  constexpr mdarray(container_type&& c, const extents_type& ext);
template<class Alloc>
  requires(uses_allocator_v<container_type, Alloc>)
  constexpr mdarray(container_type&& c, const mapping_type& m);

template<class Alloc>
  requires(uses_allocator_v<container_type, Alloc>)
  constexpr mdarray(initializer_list<T> init, const extents_type& ext);
template<class Alloc>
  requires(uses_allocator_v<container_type, Alloc>)
  constexpr mdarray(initializer_list<T> init, const mapping_type& m);

template<class OtherElementType, class OtherExtents,
         class OtherLayoutPolicy, class OtherContainer,
         class Alloc>
  explicit(see below)
  requires(uses_allocator_v<container_type, Alloc>)
  constexpr mdarray(
    const mdarray<OtherElementType, OtherExtents,
                  OtherLayoutPolicy, OtherContainer>& other);

constexpr mdarray& operator=(const mdarray& rhs) = default;
constexpr mdarray& operator=(mdarray&& rhs) = default;

// [mdspan.mdspan.members], mdspan members
template<class... SizeTypes>
  constexpr reference operator[](SizeTypes... indices);
template<class... SizeTypes>
  constexpr const_reference operator[](SizeTypes... indices) const;
template<class SizeType, size_t N>
  constexpr reference operator[](const array<SizeType, N>& indices);
template<class SizeType, size_t N>
  constexpr const_reference operator[](const array<SizeType, N>&
      indices) const;

static constexpr size_t rank() { return Extents::rank(); }
static constexpr size_t rank_dynamic() { return Extents::rank_dynamic();
      }
static constexpr size_type static_extent(size_t r) { return
      Extents::static_extent(r); }

constexpr const extents_type& extents() const { return map_.extents(); }
```

```
  constexpr size_type extent(size_t r) const { return extents().extent(r);
        }
  constexpr size_type size() const;

  constexpr pointer data() { return ctr\_.data(); }
  constexpr const_pointer data() const { return ctr\_.data(); }
  constexpr container_type& container() { return ctr\_; }
  constexpr const container_type& container() const { return ctr\_; }
  constexpr const mapping_type& mapping() const { return map_; }

  template<class OtherElementType, class OtherExtents,
           class OtherLayoutType, class OtherAccessorType>
  operator mdspan ();


  static constexpr bool is_always_unique() {
    return mapping_type::is_always_unique();
  }
  static constexpr bool is_always_contiguous() {
    return mapping_type::is_always_contiguous();
  }
  static constexpr bool is_always_strided() {
    return mapping_type::is_always_strided();
  }

  constexpr bool is_unique() const {
    return map_.is_unique();
  }
  constexpr bool is_contiguous() const {
    return map_.is_contiguous();
  }
  constexpr bool is_strided() const {
    return map_.is_strided();
  }
  constexpr size_type stride(size_t r) const {
    return map_.stride(r);
  }

private:
  container_type ctr_;
  mapping_type map_; // exposition only
};
```

4 `mdarray<ElementType, Extents, LayoutPolicy, Accessor>` is a trivially copyable type if `Container` and `LayoutPolicy::mapping_type<Extents>` are trivially copyable types.

5 `ElementType` is required to be a complete object type that is neither an abstract class type nor an array type. If `Extents` is not a specialization of `extents`, then the program is ill-formed. `LayoutPolicy` shall meet the layout mapping policy requirements. `Container` shall meet the requirements of ContiguousContainer. If `is_same_v<typename Container::value_type, ElementType>` equals `false`, then the program is ill-formed.

6 The default type for `Container` is `vector<ElementType>` if `Extents::dynamic_rank()>0` is `true`. Otherwise, the default type is `array<ElementType,N>`, where `N` is the product of `Extents::static_extent(r)` for `r` in the range of `[0, Extents::rank())`.

22.7.[illegible].1 `mdarray` Constructors [mdarray.ctors]

```
template<class... SizeTypes>
  requires(rank()>0 || rank_dynamic()==0)
  explicit constexpr mdarray(SizeTypes... exts);
```

1 *Constraints:*

(1.1) — `(is_convertible_v<SizeTypes, size_type> && ...)` is `true`,

(1.2) — `is_constructible_v<Extents, static_cast<size_type>(SizeTypes)...>` is `true`,

(1.3) — `is_constructible_v<mapping_type, Extents>` is `true`, and

(1.4) — `is_constructible_v<container_type, size_t>` is `true`.

2 *Effects:*

(2.1) — Direct-non-list-initializes *map_* with `Extents(static_cast<size_type>(std::move(exts))...)`, and

(2.2) — Direct-non-list-initializes `ctr_` with `container_type(`*map_*`.required_span_size())`.

```
constexpr mdarray(const extents_type& ext);
```

3 *Constraints:*

(3.1) — `is_constructible_v<mapping_type, const extents_type&>` is `true`, and

(3.2) — `is_constructible_v<container_type, size_t>` is `true`.

4 *Effects:*

(4.1) — Direct-non-list-initializes *map_* with `ext`, and

(4.2) — Direct-non-list-initializes `ctr_` with `container_type(`*map_*`.required_span_size())`.

```
constexpr mdarray(const mapping_type& m);
```

5 *Constraints:* `is_constructible_v<container_type, size_t>` is `true`.

6 *Effects:*

(6.1) — Direct-non-list-initializes *map_* with `m`, and

(6.2) — Direct-non-list-initializes `ctr_` with `container_type(map_.required_span_size())`.

```
template<class... SizeTypes>
  explicit constexpr mdarray(const container_type& c, SizeTypes... exts);
```

7 *Constraints:*

(7.1) — `(is_convertible_v<SizeTypes, size_type> && ...)` is `true`,

(7.2) — `is_constructible_v<extents_type, static_cast<size_type>(SizeTypes)...>` is `true`,

(7.3) — `is_constructible_v<mapping_type, extents_type>` is `true`, and

8 *Preconditions:* `c.size() == mapping_type(extents_type(static_cast<size_type>(std::move(exts))...)).required_span_size()` is `true`.

9 *Effects:*

(9.1) — Direct-non-list-initializes *map_* with `extents_type(static_cast<size_type>(std::move(exts))...)`, and

(9.2) — Direct-non-list-initializes `ctr_` with `c`.

```
constexpr mdarray(const container_type& c, const extents_type& ext);
```

10 *Constraints:* `is_constructible_v<mapping_type, const extents_type&>` is `true`, and

11 *Preconditions:* `c.size() == mapping_type(ext).required_span_size()` is `true`.

12 *Effects:*

(12.1) — Direct-non-list-initializes *map_* with `ext`, and

(12.2) — Direct-non-list-initializes `ctr_` with `c`.

```
constexpr mdarray(const container_type& c, const mapping_type& m);
```

13 *Preconditions:* `c.size() == m.required_span_size()` is `true`.

14 *Effects:*

(14.1) — Direct-non-list-initializes *map_* with `m`, and

(14.2) — Direct-non-list-initializes `ctr_` with `c`.

```
template<class... SizeTypes>
  explicit constexpr mdarray(container_type&& c, SizeTypes... exts);
```

15 *Constraints:*

(15.1) — `(is_convertible_v<SizeTypes, size_type> && ...)` is `true`,

(15.2) — `is_constructible_v<extents_type, static_cast<size_type>(SizeTypes)...>` is `true`,

(15.3) — `is_constructible_v<mapping_type, extents_type>` is `true`, and

16 *Preconditions:* `c.size() == mapping_type(extents_type(static_cast<size_type>(std::move(exts))...)).required_span_size()` is `true`.

17 *Effects:*

(17.1) — Direct-non-list-initializes *`map_`* with `extents_type(static_cast<size_type>(std::move(exts))...)`, and

(17.2) — Direct-non-list-initializes `ctr_` with `std::move(c)`.

```
constexpr mdarray(container_type&& c, const extents_type& ext);
```

18 *Constraints:* `is_constructible_v<mapping_type, const extents_type&>` is `true`, and

18 *Preconditions:* `c.size() == mapping_type(ext).required_span_size()` is `true`.

19 *Effects:*

(19.1) — Direct-non-list-initializes *`map_`* with `ext`, and

(19.2) — Direct-non-list-initializes `ctr_` with `std::move(c)`.

```
constexpr mdarray(container_type&& c, const mapping_type& m);
```

20 *Preconditions:* `c.size() == m.required_span_size()` is `true`.

21 *Effects:*

(21.1) — Direct-non-list-initializes *`map_`* with `m`, and

(22.2) — Direct-non-list-initializes `ctr_` with `std::move(c)`.

```
template<class OtherElementType, class OtherExtents,
         class OtherLayoutPolicy, class OtherContainer>
  explicit(see below)
  constexpr mdarray(const mdarray<OtherElementType, OtherExtents,
                                  OtherLayoutPolicy, OtherContainer>&
        other);
```

23 *Mandates:*

(23.1) — `is_constructible_v<Container, const OtherContainer&>` is `true`, and

(23.2) — `is_constructible_v<extents_type, OtherExtents>` is `true`.

24 *Constraints:*

(24.1) — `is_constructible_v<mapping_type, const OtherLayoutPolicy::template mapping<OtherExtents>&>` is `true`, and

(24.2) — `is_constructible_v<container_type, OtherContainer>` is `true`.

25 *Preconditions:*

(25.1) — For each rank index `r` of `extents_type`, `static_extent(r) == dynamic_extent || static_extent(r) == other.extent(r)` is `true`.

26 *Effects:*

(26.1) — Direct-non-list-initializes `ctr_` with `other.ctr_`, and

(26.2) — Direct-non-list-initializes *`map_`* with `other.`*`map_`*.

27 *Remarks:* The expression inside `explicit` is:

```
  !is_convertible_v<const typename OtherLayoutPolicy::mapping_type&,
        mapping_type> ||
  !is_convertible_v<const OtherContainer&, Container>
```

```
template<class Alloc>
  requires(uses_allocator_v<container_type, Alloc>)
  constexpr mdarray(const extents_type& ext, const Alloc& a);
```

28 *Constraints:*

(28.1) — `is_constructible_v<mapping_type, const extents_type&>` is `true`, and

(28.2) — `is_constructible_v<container_type, size_t, Alloc>` is `true`.

29 *Effects:*

(29.1) — Direct-non-list-initializes *`map_`* with `ext`, and

(29.2) — Direct-non-list-initializes `ctr_` with *`map_`*`.required_span_size()` as the first argument and `a` as the second argument.

```
template<class Alloc>
  requires(uses_allocator_v<container_type, Alloc>)
  constexpr mdarray(const mapping_type& m, const Alloc& a);
```

30 *Constraints:* `is_constructible_v<container_type, size_t, Alloc>` is `true`.

31 *Effects:*

(31.1) — Direct-non-list-initializes *`map_`* with `m`, and

(31.2) — Direct-non-list-initializes `ctr_` with `m.required_span_size()` as the first argument and `a` as the second argument.

```
template<class Alloc>
  requires(uses_allocator_v<container_type, Alloc>)
  constexpr mdarray(const container_type& c, const extents_type& ext,
        const Alloc& a);
```

32 *Constraints:*

(32.1) — `is_constructible_v<mapping_type, const extents_type&>` is `true`, and

(32.2) — `is_constructible_v<container_type, container_type, Alloc>` is `true`.

33 *Preconditions:* `c.size() == mapping_type(ext).required_span_size()` is `true`.

34 *Effects:*

(34.1) — Direct-non-list-initializes *`map_`* with `ext`, and

(34.2) — Direct-non-list-initializes `ctr_` with `c` as the first argument and `a` as the second argument.

```
template<class Alloc>
  requires(uses_allocator_v<container_type, Alloc>)
  constexpr mdarray(const container_type& c, const mapping_type& m, const
        Alloc& a);
```

35 *Constraints:* `is_constructible_v<container_type, container_type, Alloc>` is `true`.

36 *Preconditions:* `c.size() == m.required_span_size()` is `true`.

37 *Effects:*

(37.1) — Direct-non-list-initializes *`map_`* with `m`, and

(37.2) — Direct-non-list-initializes `ctr_` with `c` as the first argument and `a` as the second argument.

```
template<class Alloc>
  requires(uses_allocator_v<container_type, Alloc>)
  constexpr mdarray(container_type&& c, const extents_type& ext, const
        Alloc& a);
```

38 *Constraints:*

(38.1) — `is_constructible_v<mapping_type, const extents_type&>` is `true`, and

(38.2) — `is_constructible_v<container_type, container_type, Alloc>` is `true`.

39 *Preconditions:* `c.size() == mapping_type(ext).required_span_size()` is `true`.

40 *Effects:*

(40.1) — Direct-non-list-initializes *`map_`* with `ext`, and

(40.2) — Direct-non-list-initializes `ctr_` with `std::move(c)` as the first argument and `a` as the second argument.

```
template<class Alloc>
  requires(uses_allocator_v<container_type, Alloc>)
  constexpr mdarray(container_type&& c, const mapping_type& m, const
        Alloc& a);
```

41 *Constraints:* `is_constructible_v<container_type, container_type, Alloc>` is `true`.

42 *Preconditions:* `c.size() == m.required_span_size()` is `true`.

43 *Effects:*

(43.1) — Direct-non-list-initializes *`map_`* with `m`, and

(43.2) — Direct-non-list-initializes `ctr_` with `std::move(c)` as the first argumnet and `a` as the second argument.

```
template<class OtherElementType, class OtherExtents,
         class OtherLayoutPolicy, class OtherContainer, class Alloc>
  explicit(see below)
  constexpr mdarray(const mdarray<OtherElementType, OtherExtents,
                                  OtherLayoutPolicy, OtherContainer>&
        other,
                    const Alloc& a);
```

44 *Mandates:*

(44.1) — `is_constructible_v<Container, OtherContainer, Alloc>` is `true`, and

(44.2) — `is_constructible_v<extents_type, OtherExtents>` is `true`.

45 *Constraints:*

(45.1) — `is_constructible_v<mapping_type, const OtherLayoutPolicy::template mapping<OtherExtents>&>` is `true`, and

(45.2) — `is_constructible_v<container_type, OtherContainer, Alloc>` is `true`.

46 *Preconditions:*

(46.1) — For each rank index `r` of `extents_type`, `static_extent(r) == dynamic_extent || static_extent(r) == other.extent(r)` is `true`.

47 *Effects:*

(47.2) — Direct-non-list-initializes *`map_`* with `other.`*`map_`*, and

(47.1) — Direct-non-list-initializes `ctr_` with `other.ctr_` as the first argument and `a` as the second argument.

48 *Remarks:* The expression inside `explicit` is:

```
  !is_convertible_v<const typename OtherLayoutPolicy::mapping_type&,
        mapping_type> ||
  !is_convertible_v<const OtherContainer&, Container>
```

22.7.□.2 `mdarray` members [mdarray.members]

```
template<class... SizeTypes>
  constexpr reference operator[](SizeTypes... indices);
```

1 *Constraints:*

(1.1) — `(is_convertible_v<SizeTypes, size_type> && ...)` is `true`,

(1.2) — `(is_nothrow_constructible_v<size_type, SizeTypes> && ...)` is `true`, and

(1.3) — `sizeof...(SizeTypes) == rank()` is `true`.

2 Let `I` be `static_cast<size_type>(std::move(indices))`.

3 *Preconditions:* `I` is a multidimensional index into `extents()`. *[Note:* This implies that *`map_`*`(I...) <`*`map_`*`.required_span_size()` is `true`.— *end note]*;

4 *Effects:* Equivalent to: `return ctr_[`*`map_`*`(I...)];`.

```
template<class... SizeTypes>
  constexpr const_reference operator[](SizeTypes... indices) const;
```

5 *Constraints:*

(5.1) — `(is_convertible_v<SizeTypes, size_type> && ...)` is `true`,

(5.2) — `(is_nothrow_constructible_v<size_type, SizeTypes> && ...)` is `true`, and

(5.3) — `sizeof...(SizeTypes) == rank()` is `true`.

6 Let `I` be `static_cast<size_type>(std::move(indices))`.

7 *Preconditions:* `I` is a multidimensional index into `extents()`. *[Note:* This implies that *`map_`*`(I...) <`*`map_`*`.required_span_size()` is `true`.— *end note]*;

8 *Effects:* Equivalent to: `return ctr_[`*`map_`*`(I...)];`.

```
template<class SizeType, size_t N>
  constexpr reference operator[](const array<SizeType, N>& indices);
```

9 *Constraints:*

(9.1) — `is_convertible_v<const SizeType&, size_type>` is `true`, and

(9.2) — `is_nothrow_constructible_v<size_type, const SizeType&>` is `true`, and

(9.3) — `rank() == N` is `true`.

10 *Effects:* Let `P` be the parameter pack such that `is_same_v<make_index_sequence<rank()>, index_sequence<P...>>` is `true`. Equivalent to: `return operator[](static_cast<size_type>(indices[P])...);`

```
template<class SizeType, size_t N>
  constexpr const_reference operator[](const array<SizeType, N>& indices)
        const;
```

11 *Constraints:*

(11.1) — `is_convertible_v<const SizeType&, size_type>` is `true`, and

(11.2) — `is_nothrow_constructible_v<size_type, const SizeType&>` is `true`, and

(11.3) — `rank() == N` is `true`.

12 *Effects:* Let `P` be the parameter pack such that `is_same_v<make_index_sequence<rank()>, index_sequence<P...>>` is `true`. Equivalent to: `return operator[](static_cast<size_type>(indices[P])...);`

```
constexpr size_type size() const;
```

13 *Precondition:* The size of `extents()` is a representable value of `size_type` ([basic.fundamental]).

14 *Returns:* `extents().`*`fwd-prod-of-extents`*`(rank())`.

```
  template<class OtherElementType, class OtherExtents,
           class OtherLayoutType, class OtherAccessorType>
  operator mdspan ();
```

15 *Constraints:* `is_assignable_<mdspan<element_type, extents_type, layout_type>, mdspan<OtherElementType, OtherExtents, OtherLayoutType, OtherAccessorType>>` is `true`.

16 *Returns:* `mdspan(data(),`*`map_`*`)`

§ 5 References

[P0009R9]
H. Carter Edwards, Bryce Adelstein Lelbach, Daniel Sunderland, David Hollman, Christian Trott, Mauro Bianco, Ben Sander, Athanasios Iliopoulos, John Michopoulos, Mark Hoemmen. 2019. mdspan: A Non-Owning Multidimensional Array Reference.
https://wg21.link/p0009r9